9 Executing Language Models Locally

Large language models (LLMs) have been the talk of the town since the groundbreaking success of ChatGPT. Whether it's Gemini, Claude, or ChatGPT, they all have one thing in common: they are provided by companies, and these companies even provide the interfaces to their models free of charge. As a Pro customer, you'll then receive certain features such as priority processing of inquiries or the latest and best model variants for a relatively modest fee.

Whichever option you choose, you're always dealing with a service that is hosted in the cloud, so you don't know for sure what happens to the data that you transmit to the services via your prompts. Another criticism of these online services is that you're dependent on the respective provider. Currently, a sufficient number of offers are available so that you have no problem if a provider withdraws its offer, but the dependency remains.

Concerns about privacy and dependence on one provider in particular have led some people and companies to look for alternatives when it comes to using language models. This is why there is a growing interest in executing LLMs locally. This chapter reveals how well this works:

- First, we'll provide an overview of freely available LLMs. We also address the question of how powerful your hardware needs to be in order to run language models locally at a reasonable speed.
- Then, we describe the installation and use of the free *GPT4All* and *Ollama* programs. Both programs are compatible with a wide range of LLMs and can even take local documents into account when generating the response. While GPT4All is intended for beginners, Ollama provides a good basis for many professional applications, from coding to application programming interface (API) use.

- Ollama is actually a purely text-oriented tool. However, you can combine the program with the *Open WebUI* interface and then execute prompts similar to the web-based versions of ChatGPT and others.
- The *Continue* plug-in implements a code wizard you can use instead of GitHub Copilot or other commercial services. Continue is able to communicate with a local Ollama installation or with external LLM APIs.
- Ollama can also be used via an API. This allows you to write your own programs that don't rely on commercial LLMs, but instead use a locally executed language model.
- The *Tabby* project represents an alternative to the combination of Ollama and Continue: it consists of an LLM server, which can be installed on your own computer or on a powerful workstation in the company, and an editor plug-in with code wizard functions.

9.1 Spoiled for Choice of Large Language Model

If you deal with generative AI, you'll inevitably come across various LLMs. This section is intended to give you a rough overview of the different model families, their intended use, and the resources required for local models.

9.1.1 Which Models Solve Which Problems?

In general, you need to distinguish between commercial and freely available models for LLMs. The commercial models are provided by providers as services. ChatGPT is just one example of such a service. In the basic version, you can use ChatGPT for your requests free of charge in your browser. OpenAI, the provider behind ChatGPT, offers a monthly subscription model with several benefits, such as access to better models, higher message volumes or earlier access to new features. Anthropic is pursuing similar strategies with Claude as is Google with Gemini.

In addition to the familiar browser interface or apps, you can also use these models via an API. Here the billing model is usually different, and you have to pay per token, that is, both for input and output tokens. The fees per token are usually very low. For example, you can buy one million input tokens from OpenAI for $2.50 and one million output tokens for $10. The smaller GPT-4o Mini model costs $0.15 for one million input tokens and $0.60 for one million output tokens (as of January 2025).

There are no costs for the local version of freely available models such as Llama, Gemma, or Mistral. However, you must now provide the infrastructure on which the models are executed. For this reason, you should weigh very carefully which variant you choose. You can usually control the costs through the quality of the model and the response time.

There are currently many different LLMs on the market. Some models share the same architecture and can be grouped into families. With GPT, Llama, Mistral, and Claude, you'll get to know some examples and their respective special features in the following list:

- **GPT: Generative Pre-Trained Transformers by OpenAI**
  Probably the best-known models belong to the GPT family from OpenAI. They form the basis for ChatGPT. GPT is based on the transformer model and uses a special variant of it that is optimized for text generation. A key to the success of these models is comprehensive training with a large amount of text and subsequent fine-tuning.
  OpenAI provides the GPT models via its own services. Its use is partly free of charge and partly subject to charge. The models in the GPT family can be multimodal; that is, they can work not only with text but also with images or audio. Unfortunately, the LLM files from OpenAI can't be downloaded for free.
  Typical tasks include chatbots, content generation, and code generation.
- **Llama: Large Language Model Meta AI**
  Llama, the language models from Meta, use a similar architecture to GPT and are based on the transformer architecture. Thanks to various optimizations and design decisions, the focus of these models is more on efficiency and performance with a smaller model size.
  Meta's Llama models are subject to the Llama Community License Agreement. This license is intended for private use and research, but restricts the commercial use of the models.
  Typical tasks include natural language processing (NLP) tasks such as text generation, translation, and research tasks. Code Llama is a variant of Llama optimized for coding tasks.
- **Mistral/Mixtral**
  The Mistral and Mixtral models are made by Mistral, a French company specializing in AI. The company is known for its open-source models,

which are subject to the Apache 2 license. In addition to these, Mistral also offers other paid models via its own APIs.
Mixtral, the latest models from Mistral, are Mixture-of-Experts Language Models (MoE-LLMs) which combine several expert models and thus achieve better performance and a high degree of accuracy. In contrast to GPT and Llama, Mixtral isn't a complete transformer model, but a pure decoder model.
Typical tasks include summarizing texts, structuring, answering questions, and code completion.

- **Claude**
  The Claude models are developed by Anthropic. The focus of development is not only on the performance of the models but also on safety and ethics.
  At their core, the Claude models are also based on a transformer architecture. The difference lies in the constitutional AI, a special training method based on supervised learning and reinforcement learning.
  Another special feature of the modern Claude models is their context size of 200,000 tokens, which is very large compared to the competition. Unfortunately, like OpenAI, Anthropic doesn't offer a download option for its models.
  Typical tasks include commercial applications, complex analyses, summarizing large amounts of data, and coding.

An objective comparison of which language model delivers the ideal result for which application is extremely difficult. There is no generally accepted standard for this yet, but there are numerous projects and scientific studies. Reading the following, selected with regard to coding applications, will give you a first impression:

- **https://lmarena.ai/?leaderboard**
- **https://artificialanalysis.ai/models/gpt-4o/providers**
- **https://github.com/continuedev/what-llm-to-use**
- **https://evalplus.github.io/leaderboard.html**
- **https://symflower.com/en/company/blog/2024/comparing-llm-benchmarks**

Ultimately, you'll have to experiment yourself to find out which LLM works best on your hardware and for your requirements.

Free or Truly Open Source?

"Free" LLMs such as Llama, Gemma, StarCoder, or Mistral are subject to separate licenses, which may restrict their use. Most LLMs have more restrictions than you're used to from the Linux world, for example.

Especially before you start using free LLMs commercially, you must familiarize yourself with the terms of use. These can be found on the respective project page and on **https://huggingface.co** in the description of the respective language model.

9.1.2 Hardware Requirements

Being able to download numerous language models free of charge is great, but the local execution of such models using programs such as GPT4All or Ollama often fails due to the high hardware requirements. For example, if you try to run the Llama 3.1 model in the best quality 405B version on a standard computer, it's simply not possible. For this reason, we'll now take a look at the general hardware requirements for running an LLM:

- **CPU**
  You should have at least an 11th generation Intel chip or a Zen4-based AMD CPU. Apple's M-CPUs are also very suitable. More important than the sheer number of cores is the feature set of the processors, especially the accelerated matrix multiplication required by AI models.
- **RAM**
  For a 7B model, your system should have at least 16 GB of RAM. (7B means 7 billion parameters.)
- **Hard disk space**
  You should plan about 50 GB of free space on your hard disk so that you can save a few models to try out.
- **Graphics processing unit (GPU)**
  Although a graphics card isn't absolutely necessary, it speeds up the execution of the models considerably. However, the entire model must be

stored in the video memory of the graphics card. For a 4-bit quantized model, you should expect the following VRAM sizes:

- 7B: about 4 GB VRAM
- 13B: about 8 GB VRAM
- 30B: about 16 GB VRAM
- 65B: about 32 GB VRAM

Apple computers with ARM CPUs (Apple Silicon) are a special case. Their GPUs can access the entire working memory, so there is no need to differentiate between RAM and VRAM.

These recommendations are from Justin Hayes, one of the contributors to Open WebUI and Ollama, and can be found in a GitHub issue of Open WebUI here:

https://github.com/open-webui/open-webui/discussions/736#discussioncomment-8474297

As long as you provide your platform with sufficient memory so that it can load the corresponding model, you can also run it. However, if you use hardware that is too weak, you must expect the model's feedback to be very slow. It's better to use a slightly smaller model that works smoothly on your system rather than a model that can just about run on your system.

Speed Benchmarks

A decisive parameter for estimating how well the local execution of a language model works is the number of tokens that can be processed (input) or generated (output) per second. For interactive use in coding, your local language model should achieve at least 30 tokens/s (output).

The token rate depends primarily on two things: the model size and the available hardware. On the following web pages, you'll find results for various LLMs and hardware configurations on Windows, Linux, and macOS:

- **https://llm.aidatatools.com/results-windows.php**
- **https://llm.aidatatools.com/results-linux.php**
- **https://llm.aidatatools.com/results-macos.php**

9.1.3 Alternatives to the Local Execution of Models

In addition to the local execution of AI models and the use of commercial models, there is another intermediate step. You can also rent virtual machines with GPUs in the cloud. All major providers offer such services. Note, however, that the use of such virtual machines can sometimes incur considerable costs, so you should carefully consider whether you only need them occasionally. The following overview shows a selection of options for such platforms from the major cloud providers:

- **Amazon Web Services (AWS): EC2 GPU Instance**
  https://aws.amazon.com/ec2/instance-types/#Accelerated_Computing
- **Google Cloud Platform (GCP): Cloud GPU**
  https://cloud.google.com/gpu
- **Microsoft Azure: NVIDIA Deep Learning (ND) or NVIDIA Compute (NC) family compute instances**
  https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/overview
- **NVIDIA: GPU cloud computing**
  www.nvidia.com/en-us/data-center/gpu-cloud-computing/

The various providers offer cloud instances in different sizes. The prices depend heavily on the respective configuration and the term. For example, if you rent an *NCasT4_v3* instance of Microsoft Azure, it will cost you around $500 per month. In return, you get 28 GB RAM, 176 GB hard disk space, and an NVIDIA Tesla T4 GPU with 16 GB VRAM.

You can increase this configuration even further if you select a *NC48ads_A100_v4* instance, for example. This instance offers you 880 GB RAM, 96 Cores, 3,832 Temporary Storage, and 4 NVIDIA A100 GPUs with 80 GB VRAM each. With this configuration, however, the costs rise to around $14,000 per month!

For an exact price calculation, you should consult the price calculator of the respective provider. No matter which commercial provider you choose, there will always be costs for you. An alternative is local execution, either on your own computer or in your local network. For this purpose, you can use platforms such as GPT4All or Ollama, which make it easier for you to run models.

Groq

With its GroqCloud, the company Groq (not to be confused with the AI chatbot Grok from xAI!) is treading a middle path between commercial AI API providers such as OpenAI or Anthropic and pure cloud/hardware providers such as AWS. Groq enables you to execute LLM prompts via API keys. Various open-source LLMs in different sizes are available to choose from. Billing is based on usage (number of tokens with different prices depending on the model). Groq uses an internally developed AI accelerator (*Language Processing Unit*) that can execute LLMs particularly quickly. You can find information on this topic at **https://groq.com/groqcloud**.